Performance Engineering

Lab 7

Jort Oosterveld S5545781 & Sergiu Preda S6778542
Group 11

October 21, 2026

1 Data Structure Optimization

```
1  const int n = 16384;
2
3  struct Small {
4      char flg1;
5      double x[7];
6      char flg2;
7  };
8
9  struct Big {
10     int rank;
11     double x[n];
12 };
```

Listing 1: Data structures Small and Big

Both structures are sorted with `std::sort`; the comparator reads one key field (`x[0]` resp. `rank`) and the swaps move the whole records.

1.1 a.

Which of the following techniques can be used for optimizing the data structures `Small` and `Big`?

- **Data packing**
  - **Small:** No, merging the flags into one bit-field saves one byte, which the 8-byte alignment of `x` undoes, making the structure 64 B again. Forcing packed chars misaligns the doubles, which reduces performance.
  - **Big:** No, but `rank` could be a 16-bit `short`, because its value is always $< n$, which saves only 2 bytes in a huge structure.
- **Data padding**
  - **Small:** Yes, if we use a 64B align, so that every record is exactly one line instead of parts of two different records.
  - **Big:** Yes, small performance increase from aligning to 64B.
- **Field re-ordering**
  - **Small:** Yes, if we move the flags, the holes in the structure disappear
  - **Big:** No. Moving `rank` behind the array exchanges the 4B hole for 4B of tail padding, so the size stays the same.

- **Structure splitting**
  - **Small:** Yes, the sort only reads `x[0]`, so a hot array of (`x[0]`, index) pairs (16 B) can be sorted and the permutation applied to the 64/72B records once. The swaps then move 16 instead of 72 B, at the price of one extra pass and a changed layout for the rest of the program. We expect to see a small gain, since the whole vector fits in L1/L2 already.
  - **Big:** Yes, the comparator reads only 4B out of a 128B record, having to fetch a new page every time. Every swap copies 3×128KiB. Splitting `rank` (plus an index) from the payload lets the sort work on only the small keys, instead of the whole structure. The payload is permuted once afterwards.

1.2 B.

We combine the techniques that a) identified as useful: for `Small` field re-ordering, cache-line alignment and structure splitting, and for `Big` cache-line padding and structure splitting (Listing 2).

```
using Key = double;

// 64 B, one line per record
struct alignas(64) Small {
    double x[7];
    char flg1;
    char flg2;
};
// cache-line aligned
struct alignas(64) Big {
    int rank;
    double x[n];
};

// hot part: 16 B (Small), 8 B (Big)
struct KeyIdx {
    Key k;
    uint32_t i;
};

std::vector<KeyIdx> ki(cnt);
for (uint32_t i = 0; i < cnt; i++) {
    ki[i] = {key(v[i]), i};
}

std::sort(ki.begin(), ki.end(), [](auto &a, auto &b) { return a.k < b.k; });

// position p must receive record ki[p].i: follow every cycle with one temporary
std::unique_ptr<Rec> tmp(new Rec);
for (uint32_t p = 0; p < cnt; p++) {
    if (done[p] || ki[p].i == p) {
        done[p] = 1;
        continue;
    }
    memcpy(tmp.get(), &v[p], sizeof(Rec));
    for (uint32_t cur = p;;) {
        uint32_t src = ki[cur].i;
        done[cur] = 1;
        if (src == p) {
            memcpy(&v[cur], tmp.get(), sizeof(Rec));
            break;
        }
        memcpy(&v[cur], &v[src], sizeof(Rec));
```

```
            cur = src;
        }
}
```

Listing 2: Snippets of the optimized layouts and the split sort. `v[]` is the original array of either Big or Small, of type `Rec`. `cnt` is the length of `v[]`, ,

The sort works on the hot array of (key, index) pairs. Carrying the original index in the hot part keeps the link to the cold part (the unchanged record vector) at a cost of 4 B per record, and lets the sort itself move only 16 B (`Small`) or 8 B (`Big`) per swap. The permutation step then makes the end state identical to the original benchmark, so the comparison is fair.

1.3 C.

Setup: One cn-ib node of HAICGU (cn15, Kunpeng 920 at 2.6 GHz: 64 KB L1d, 512 KB private L2, 32 MB L3, 64 KiB pages), GCC 8.5.0 `-O2 -mcpu=native`, `perf` 4.18, with the process pinned to one core.
Every configuration runs three times with two event groups of at most six events, so that `perf` does not have to make estimates by multiplexing counters. The results were all within a margin of $< 1\%$ of each other. The counters of a run with R sorts minus those of a run with $R = 0$ (allocate and initialise only), divided by R, give the values per sort below. "L3 reads/misses" are the ARM events `ll_cache_rd` and `ll_cache_miss_rd`.

per sort	base	optimized
record size [B]	72	64
time per sort [ms]	0.459	0.342
of which key sort [ms]	–	0.249
of which permutation [ms]	–	0.0931
speed-up vs. base	1.00	1.34
cycles	1.19 M	891 k
instructions	2.03 M	1.9 M
L1d loads	951 k	724 k
L1d load misses	38.5 k	9.62 k
L2 accesses	293 k	187 k
L2 refills	54.3 k	37.3 k
L3 reads	54 k	34.2 k
L3 read misses	17.5	19.8
dTLB load misses	3.75	19.5
backend stall cycles	266 k	218 k
branch misses	2.65 k	2.65 k
IPC	1.71	2.13
L1d miss rate [%]	4.1	1.3
backend stall [% cycles]	22	24

Table 1: `Small`: counters per sort of 16 384 records (median of 3 trials, reps=0 run subtracted).

Small, 16 384 records (Table 1): The vector fully fits in L1/L2, so L3 read misses and dTLB misses are zero for both versions, and the baseline is not DRAM-bound. The optimized version is therefore only 1.34× faster. The sort itself now touches 256 KB of 16-B key/index pairs instead of 1.2 MB of records, so the L1d misses drop 4× and the L2 refills by 31 %. The 64-B, line-aligned records make the permutation cheap because each record is one full cache line and no copy crosses a line boundary. L1d loads drop by 24 % and instructions by 7 %: the data moved per swap shrinks from 72 to 16 B, and the final permutation moves every record only once.
Small, 2^{20} records (Table 2): The vector now exceeds L3 cache, and the performance increase grows to 1.77×. The baseline reads 209 MB from DRAM per sort. In the optimized version the 16-MB key array stays in L3 while it is sorted and the permutation is the only pass that streams

per sort	base	optimized
record size [B]	72	64
time per sort [ms]	64	36.2
of which key sort [ms]	–	27
of which permutation [ms]	–	9.24
speed-up vs. base	1.00	1.77
cycles	167 M	95.4 M
instructions	166 M	154 M
L1d loads	67.3 M	52.9 M
L1d load misses	4.47 M	783 k
L2 accesses	32.6 M	17.6 M
L2 refills	10.8 M	4.56 M
L3 reads	10.8 M	4.22 M
L3 read misses	3.27 M	2.06 M
dTLB load misses	8.71 k	4.66 k
backend stall cycles	31.1 M	39.7 M
branch misses	157 k	149 k
IPC	0.99	1.62
L1d miss rate [%]	6.6	1.5
backend stall [% cycles]	19	42

Table 2: `Small`: counters per sort of 1 048 576 records (median of 3 trials, reps=0 run subtracted).

per sort	base	optimized
record size [B]	131080	131136
time per sort [ms]	936	287
of which key sort [ms]	–	0.427
of which permutation [ms]	–	287
speed-up vs. base	1.00	3.26
cycles	2.43 G	748 M
instructions	1.56 G	506 M
L1d loads	1.24 G	404 M
L1d load misses	16.9 M	6.61 M
L2 accesses	605 M	177 M
L2 refills	116 M	42.4 M
L3 reads	93.9 M	34.1 M
L3 read misses	67.1 M	32.6 M
dTLB load misses	65.9 k	19.2 k
backend stall cycles	1.81 G	545 M
branch misses	83.8 k	28.5 k
IPC	0.64	0.68
L1d miss rate [%]	1.4	1.6
backend stall [% cycles]	74	73

Table 3: `Big`: counters per sort of 16 384 records (median of 3 trials, reps=0 run subtracted).

the records from DRAM. L3 read misses decrease by 37 %, the remaining 132 MB being the permutation and the part of the key sort that also needs L3 cache.
Big (Table 3): The 2 GiB vector is many times bigger than L3. Instructions and L1d loads are the copy loops, and the time follows them. The baseline reads 67M times from RAM. The optimized version sorts the 128 KB of (`rank`, index) pairs in 0.43 ms, 2000× faster than the sort of records. Permuting the records takes 287 ms. It reads every record exactly once, so only reads from DRAM half the time of the baseline. The optimized code writes only once as well, so instructions and L1d loads decrease by 3.1×. The L2 accesses drop 3.4×, more than the loads, because the padded, line-aligned records are copied without any access spanning more than a cache line. The total speedup is about 3.3×. A consumer that can work through the index array would skip the permutation and get the full 2000×. dTLB misses are small in both versions thanks to the 64 KiB pages (one record spans three pages) and still fall 3.4×. The remaining counters are left in for completeness.

2 MPI Point-To-Point Communication Benchmarking

2.1 a.

The benchmark is started as

```
mpiexec -n N ./mpi_ring [M] [trials] [sizes...]
```

so N is whatever `mpiexec` provides. Rank i sends n bytes to rank $(i+1) \bmod N$ and receives n bytes from rank $(i-1) \bmod N$. Both are posted in one `MPI_Sendrecv`, so the ring cannot deadlock for any message size (a blocking `MPI_Send` in every rank first could, once the messages are too large to be buffered).

```
right = (rank + 1) % nranks;  left = (rank - 1 + nranks) % nranks;

for (long k = 0; k < iters; k++)
    MPI_Sendrecv(sbuf, size, MPI_BYTE, right, 0,
                 rbuf, size, MPI_BYTE, left,  0, MPI_COMM_WORLD,
    MPI_STATUS_IGNORE);

MPI_Barrier(MPI_COMM_WORLD);                          // all ranks start together
double dt = MPI_Wtime();
exchange(size, M);                                    // the loop above, M times
dt = MPI_Wtime() - dt;
MPI_Allreduce(&dt, &T, 1, MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD);   // slowest
    rank
bw = (double)size * M / T / 1e6;                      // MB/s sent per rank
```

Listing 3: Timed part of the ring benchmark (all ranks execute it).

The time T of M exchanges is the maximum over the ranks, and the bandwidth is the payload one rank sends per second, $n \cdot M/T$ (1 MB = 10^6 B). Every rank receives the same amount at the same time, so each link direction carries this rate. The time per exchange T/M is reported as well, because for messages this small the latency is what is being measured. Before timing, every buffer is touched and 1000 exchanges warm up the connection. M is chosen per message size such that one run lasts about 0.2 s ($4 \cdot 10^4$ to $7 \cdot 10^5$ exchanges here), and the whole measurement is repeated 10 times. One exchange of the largest message is also checked for the right content.

2.2 b.

Setup: Four jobs (`--exclusive`, `mpiexec --bind-to core`, Open MPI 5.0.3, GCC 14.1.0, `-O2`) with the Slurm options of the assignment, on partition `cn-ib` and `cn-eth` (part c): `--nodes=1 --ntasks-per-node=2` (cn15 resp. cn20) and `--nodes=2 --ntasks-per-node=1` (cn16/cn17 resp.

cn20/cn21). The results are in Table 4 and Figure 1. The 10 repetitions of a run lie within 2.5 % of the median (min–max, all sizes), so the medians are used.

	cn-ib, same node		cn-ib, two nodes		cn-eth, same node		cn-eth, two nodes	
size [B]	μs	MB/s	μs	MB/s	μs	MB/s	μs	MB/s
128	0.29	435	1.33	96	0.30	432	2.04	63
256	0.32	805	1.77	145	0.32	791	2.46	104
512	0.39	1324	1.91	268	0.40	1282	2.60	197
1024	0.53	1936	2.27	450	0.55	1851	2.95	347
2048	0.77	2644	2.98	686	0.83	2457	3.72	550
4096	1.13	3610	4.01	1022	1.16	3544	4.87	841

Table 4: Ring benchmark with two ranks (`MPI_Sendrecv`): median time per exchange in μs and bandwidth, i.e. payload one rank sends per second, in MB/s (10^6 B/s). Median of the 10 trials per message size.

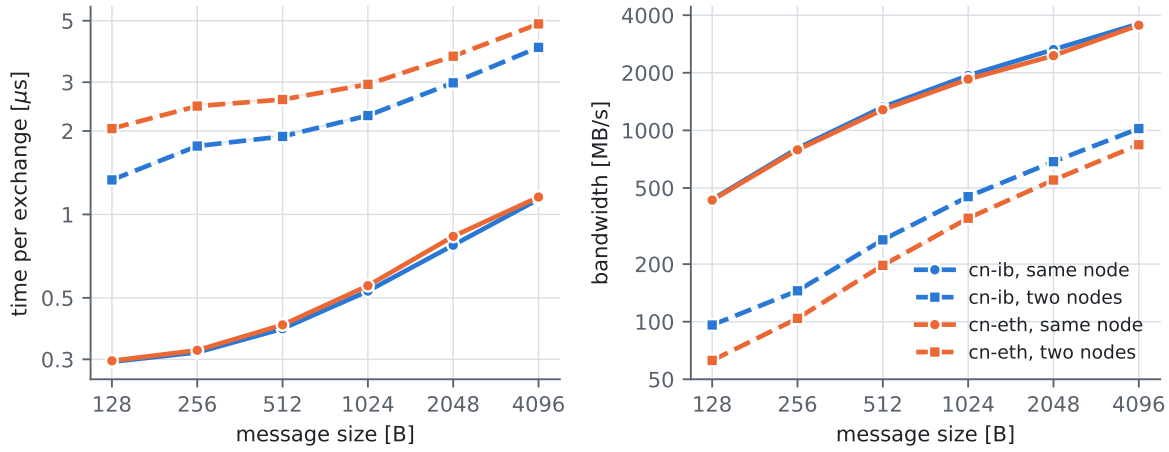


Figure 1: Time per exchange and bandwidth of the ring benchmark with two ranks. The bands are the min–max over the 10 repetitions. Both plots are logarithmic.

Message size: In this range the exchange is dominated by a fixed cost per message, not by the transfer of the data. A message that is 32× larger (128 to 4096 B) takes only 2.4× to 3.9× longer, depending on the configuration, so the bandwidth grows almost proportionally with the size: from 96 to 1022 MB/s between two nodes on `cn-ib`, and from 435 to 3610 MB/s on one node.
A straight line $t(n) = \alpha + n/\beta$ through the medians gives a start-up cost of $\alpha \approx 0.3\,\mu$s on one node and 1.5 μs (`cn-ib`) and 2.2 μs (`cn-eth`) between nodes. At 4096 B the measured bandwidth is still below the fitted asymptotic rates β (1.5–1.6 GB/s between nodes, 4.5–4.7 GB/s within a node).

Same node vs. two nodes: Communication within a node is 4.5× (cn-ib) to 6.9× (cn-eth) faster for 128 B messages and still 3.5× to 4.2× at 4096 B. A ping between two cores costs 0.29 μs, where no network is involved, while a message that has to leave the node costs at least 1.3 μs, as it has to pass the network card and the switch. At 4096 B the bandwidth is 3.6 GB/s within a node, but only 1.0 GB/s between nodes, which is less than 10 % of the 12.5 GB/s (100 Gb/s) line rate of the network. The link is therefore not the limit in this message size range; a bandwidth measurement of the network would need messages of several KiB or more.

2.3 c. (bonus)

Both partitions use the same software path: the log of every run (`ompi_info`, `pml_base_verbose`) shows that Open MPI selected the UCX component for all four jobs. The nodes also have the same NIC. `cn-ib` nodes use it through the InfiniBand interface `ib0`, `cn-eth` nodes through an Ethernet interface (`eth4`). The link is RoCE.

- **Within a node**: Both partitions are identical within measurement error (within 8 % in time per exchange, e.g. 3610 vs. 3544 MB/s at 4096 B), as expected, because no network is involved.
- **Between two nodes**: `cn-ib` latency is 0.7–0.9 μs lower for every message size (1.33 vs. 2.04 μs at 128 B, 4.01 vs. 4.87 μs at 4096 B). The offset is nearly constant, i.e. it is a cost per message, not per byte. Relatively, `cn-eth` is 1.53× slower at 128 B, but only 1.22× at 4096 B, and the bandwidth of the two networks approaches each other for larger messages (96 vs. 63 MB/s at 128 B, 1022 vs. 841 MB/s at 4096 B).

Each configuration was measured on one pair of nodes in one job, and the two partitions are measured on different nodes, so the repetitions show that the measurement is reproducible in this job, but not the spread between node pairs or between jobs.